

Diagrammes UML

Diagramme de cas d'utilisation & diagramme de classes

Module de gestion des bénévoles et des événements

Extension du site web vitrine de Terra Sana ASBL

Document complémentaire au cahier des charges (déjà transmis). Il rassemble les deux diagrammes demandés ainsi qu'une analyse détaillée, en particulier pour le diagramme de classes.

Encadreur scolaire : Marie-Christine Namur

Maître de stage : Didier Seraye

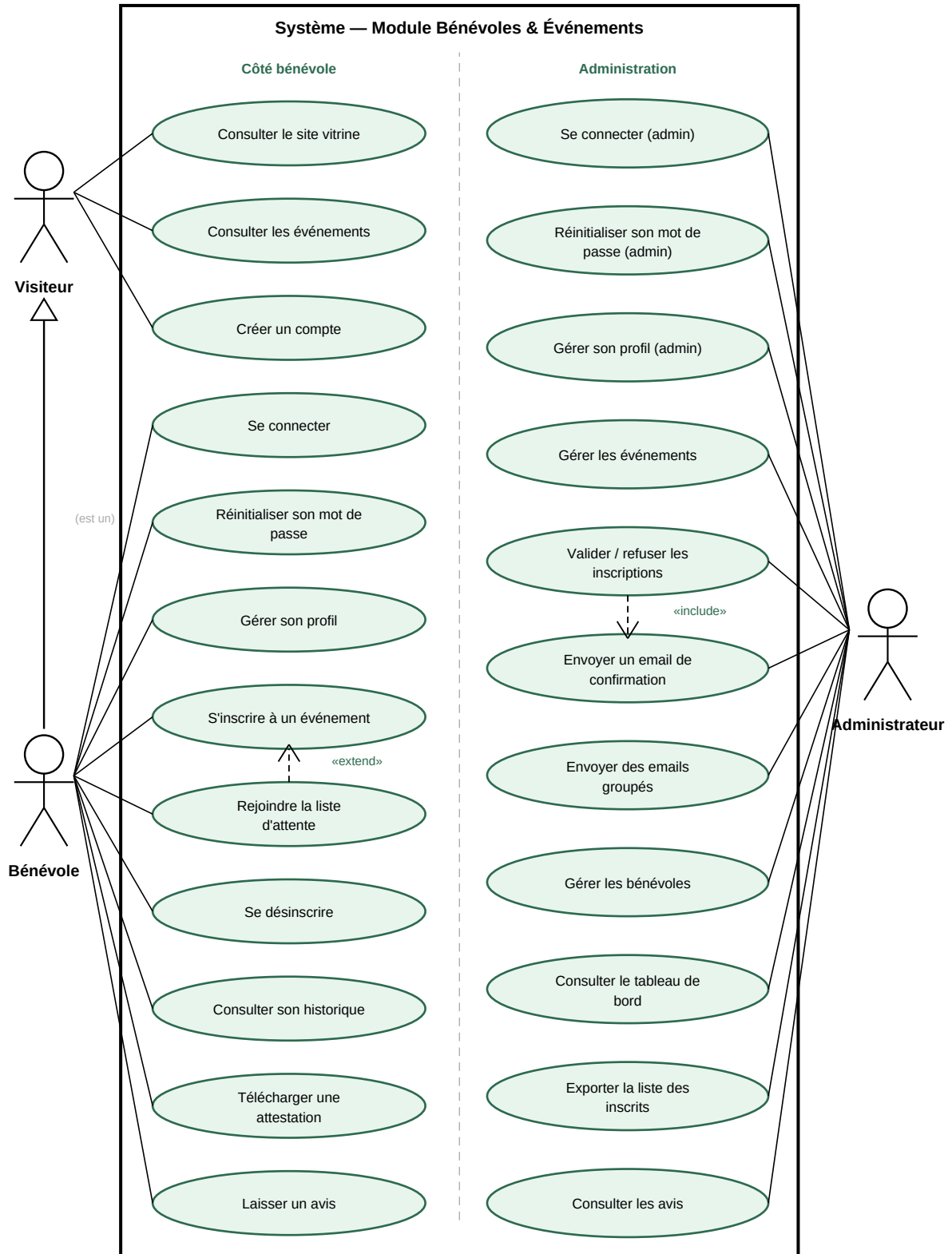
Travail présenté par Youndjeu Tchouapi Alain

EAFC Uccle

2025-2026

1. Diagramme de cas d'utilisation

Le diagramme ci-dessous décrit les interactions entre les acteurs et le module. Il distingue le périmètre côté bénévole du périmètre d'administration, tout en restant dans une seule frontière système puisqu'il s'agit d'une même application.



1.1 Les acteurs

Trois acteurs interagissent avec le module :

- **Visiteur** — internaute non authentifié. Ses seuls cas d'utilisation sont : consulter le site vitrine, consulter la liste des événements et créer un compte. Dès qu'il se connecte, il devient bénévole ; « Se connecter » et « Réinitialiser son mot de passe » sont donc rattachés au Bénévole, pas au Visiteur.
- **Bénévole** — visiteur qui s'est authentifié. Le lien de **généralisation** (flèche à triangle creux) indique qu'un bénévole *est un* visiteur : il hérite de ses cas d'utilisation (consulter le site et les événements, créer un compte) et y ajoute les siens : se connecter, réinitialiser son mot de passe, gérer son profil, s'inscrire à un événement, rejoindre la liste d'attente, se désinscrire, consulter son historique, télécharger une attestation, laisser un avis.
- **Administrateur** — membre de Terra Sana qui gère le module depuis l'espace d'administration. Il dispose de ses propres cas d'authentification et de gestion de compte (se connecter, réinitialiser son mot de passe, gérer son profil) et pilote le module : gestion des événements, validation des inscriptions, communication, gestion des bénévoles, consultation des avis, tableau de bord et export.

1.2 Les relations «include» et «extend»

Deux relations de dépendance enrichissent le modèle et traduisent des règles métier concrètes :

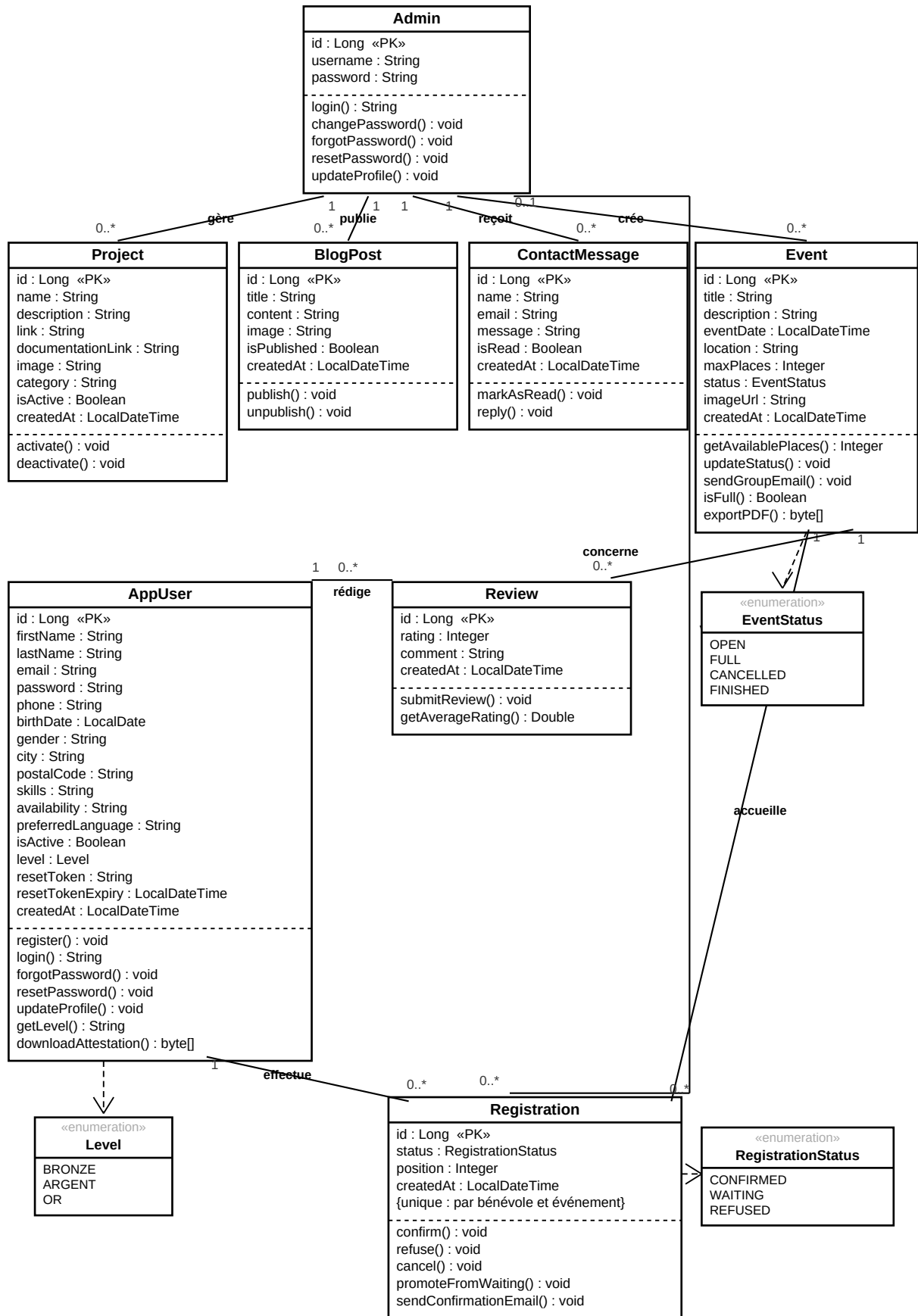
- **«extend»** — « Rejoindre la liste d'attente » **étend** « S'inscrire à un événement ». C'est un comportement *optionnel*, déclenché uniquement sous condition : lorsque l'événement a atteint son nombre maximum de places, le bénévole est basculé en liste d'attente au lieu d'être inscrit directement.
- **«include»** — « Valider / refuser les inscriptions » **inclut** « Envoyer un email de confirmation ». L'envoi de cette notification au bénévole concerné fait *systématiquement* partie du traitement : il n'y a pas de validation ou de refus sans email de confirmation. La relation « include » exprime précisément ce comportement obligatoire et réutilisable.

« Envoyer des emails groupés » est en revanche un cas d'utilisation *indépendant* : l'administrateur peut, à tout moment, notifier l'ensemble des bénévoles inscrits à un événement (rappel, changement de lieu, annulation), sans que cette action soit liée à une validation d'inscription précise.

Rappel de notation : trait plein + triangle creux = généralisation ; flèche en pointillés + pointe ouverte = dépendance «include» / «extend».

2. Diagramme de classes

Le diagramme présente les huit entités du modèle de données avec leurs attributs, leurs méthodes et leurs associations nommées. Les liens entre classes sont représentés uniquement par les associations (les clés étrangères ne sont donc pas affichées comme attributs). AppUser, Event, Review et Registration sont créées dans le cadre du TFE ; Admin, Project, BlogPost et ContactMessage proviennent du site existant.



2.1 Rôle de chaque classe

Classe	Rôle dans le module
AppUser	Représente un bénévole. Entité centrale du TFE : porte l'identité et le profil complet (coordonnées, compétences, disponibilités), l'authentification (connexion, mot de passe oublié / réinitialisé), le niveau de fidélité (Bronze / Argent / Or) et le téléchargement de l'attestation.
Event	Un événement organisé par Terra Sana : date, lieu, nombre de places, statut (ouvert / complet / annulé / clôturé), visuel. Seconde entité centrale du module.
Registration	Entité associative entre AppUser et Event. Elle matérialise l'inscription et porte ses propres attributs : statut (RegistrationStatus : CONFIRMED / WAITING / REFUSED) et position dans la file d'attente. Une association « valide » avec Admin (0..1) trace l'administrateur qui l'a traitée.
Review	Un avis laissé par un bénévole sur un événement auquel il a participé (note de 1 à 5 et commentaire).
Admin	Compte d'administration existant. Il gère l'ensemble du module (événements, inscriptions, bénévoles, avis, tableau de bord) et les contenus déjà présents sur le site (Project, BlogPost, ContactMessage), tout en assurant la gestion de son propre compte (connexion, mot de passe, profil).

2.2 Attributs, énumérations et méthodes

- **Identifiants** — chaque classe possède un identifiant `id` : Long «PK». Les liens entre classes ne sont pas dupliqués sous forme d'attributs « clé étrangère » : ils sont portés par les associations nommées du diagramme (« effectue », « accueille », etc.).
- **Énumérations** — trois attributs prennent leurs valeurs dans un ensemble fermé et sont typés par une énumération dédiée. Chaque énumération est représentée par sa propre boîte «enumeration» reliée à la classe utilisatrice par une flèche de dépendance (pointillés + pointe ouverte) : `EventStatus` (OPEN / FULL / CANCELLED / FINISHED) pour `Event . status`, `RegistrationStatus` (CONFIRMED / WAITING / REFUSED) pour `Registration . status`, et `Level` (BRONZE / ARGENT / OR) pour `AppUser . level`. Ce typage rend les valeurs autorisées explicites et évite les chaînes libres.
- **Méthodes métier** — `confirm()`, `refuse()`, `promoteFromWaiting()` et `sendConfirmationEmail()` sur `Registration` ; `getAvailablePlaces()`, `isFull()` et `exportPDF()` sur `Event` ; `downloadAttestation()` et `getLevel()` sur `AppUser`.
- **Sécurité (implémentation)** — les mots de passe ne sont jamais stockés en clair (hachage BCrypt côté Spring Security) et l'email de `AppUser` est unique en base ; ces règles sont portées par les contraintes de la base et la couche service.

Les méthodes de `Project`, `BlogPost` et `ContactMessage` (`activate()`, `publish()`, `reply()`, etc.) appartiennent à des fonctionnalités du site déjà existantes, hors du périmètre « Module Bénévoles & Événements » : c'est pourquoi elles n'apparaissent pas dans le diagramme de cas d'utilisation, qui ne couvre que le nouveau module.

2.3 Associations et cardinalités

Chaque association est nommée par un verbe métier et porte ses cardinalités. Associations propres au module TFE :

Association	Verbe	Card.	Lecture
AppUser → Registration	effectue	1 → 0..*	Un bénévole effectue plusieurs inscriptions ; chaque inscription appartient à un seul bénévole.
Event → Registration	accueille	1 → 0..*	Un événement accueille plusieurs inscriptions ; chaque inscription concerne un seul événement.
AppUser → Review	rédige	1 → 0..*	Un bénévole peut rédiger plusieurs avis.
Review → Event	concerne	0..* → 1	Un avis concerne un seul événement ; un même événement peut en recevoir plusieurs.
Admin → Registration	valide	0..1 → 0..*	Une inscription est traitée par au plus un administrateur (zéro tant qu'elle est en attente) ; un administrateur peut traiter plusieurs inscriptions.

L'administrateur, lui, pilote aussi bien les nouvelles entités que les contenus existants : il **crée** les Event (1 → 0..*), **valide** les Registration (0..1 → 0..*, puisqu'une inscription en attente n'a encore été traitée par personne), et **gère** / **publie** / **reçoit** respectivement les Project, BlogPost et ContactMessage du site. L'association **Admin — Registration** rend explicite le lien entre l'administrateur et la validation des inscriptions, plutôt que de le laisser seulement implicite via Event.

*Le verbe « **accueille** » remplace ici « reçoit » pour Event — Registration, afin de ne pas réutiliser le même mot que pour Admin — ContactMessage et éviter toute ambiguïté de lecture.*

La classe **Registration** est le pivot du modèle : placée entre AppUser et Event, elle transforme une relation « plusieurs à plusieurs » (un bénévole s'inscrit à plusieurs événements, un événement accueille plusieurs bénévoles) en deux relations « un à plusieurs » exploitables, tout en offrant un emplacement naturel pour stocker le statut et la position en liste d'attente. La contrainte {unique : par bénévole et événement} garantit qu'un même bénévole ne peut s'inscrire qu'une seule fois au même événement.

2.4 Choix de conception

- **Réutilisation de l'existant** — le compte Admin déjà présent pilote aussi bien les nouvelles entités que les contenus du site (Project, BlogPost, ContactMessage). On étend la base sans la réécrire ni dupliquer un compte de gestion. L'attribut « rôle » a été retiré : cette classe reprend déjà tous les comptes administrateur, sans notion de rôle multiple.
- **Associations plutôt que clés étrangères** — les liens entre classes sont représentés uniquement par les associations (cardinalités et verbes métier). Les clés étrangères, redondantes au niveau conceptuel, ne sont plus affichées comme attributs — elles resteront bien sûr présentes dans le schéma MySQL généré.
- **Énumérations pour les statuts et le niveau** — les valeurs fermées (statuts d'événement et d'inscription, niveau de fidélité) sont typées par des énumérations dédiées, plutôt que des chaînes libres. Cela renforce la validité des données et facilite la maintenance.
- **Cohérence avec le code** — chaque classe correspond à une entité JPA (Spring Boot) et à une table MySQL ; les attributs, types et méthodes reflètent directement l'implémentation.

3. Cohérence entre les deux diagrammes

Chaque cas d'utilisation du diagramme 1 se traduit par une ou plusieurs opérations sur les classes du diagramme 2. Le tableau ci-dessous établit cette correspondance de façon exhaustive.

Cas d'utilisation	Classe(s) / méthode(s)
Créer un compte	AppUser.register()
Se connecter	AppUser.login()
Réinitialiser son mot de passe	AppUser.forgotPassword() / resetPassword()
Gérer son profil	AppUser.updateProfile() ; AppUser.getLevel() (niveau affiché)
S'inscrire à un événement	Registration (création) ; Event.getAvailablePlaces() / isFull()
Rejoindre la liste d'attente	Registration (status = en attente, position)
Se désinscrire	Registration.cancel()
Consulter son historique	Registration (lecture, filtrée par bénévole)
Télécharger une attestation	AppUser.downloadAttestation()
Laisser un avis	Review.submitReview()
Se connecter (admin)	Admin.login()
Réinitialiser son mot de passe (admin)	Admin.forgotPassword() / resetPassword()
Gérer son profil (admin)	Admin.updateProfile() ; Admin.changePassword()
Gérer les événements	Event (création / modification) ; Event.updateStatus()
Valider / refuser les inscriptions	Registration.confirm() / refuse() ; association Admin — Registration (« valide »)
Envoyer un email de confirmation	Registration.sendConfirmationEmail()
Envoyer des emails groupés	Event.sendGroupEmail()
Gérer les bénévoles	AppUser (consultation, filtres)
Consulter le tableau de bord	Agrégation sur Event, Registration et Review
Exporter la liste des inscrits	Event.exportPDF()
Consulter les avis	Review (lecture) ; Review.getAverageRating()

Seuls « Consulter le site vitrine » et « Consulter les événements » n'apparaissent pas dans le tableau : ce sont de simples consultations (pages publiques, lecture d'Event) qui ne correspondent à aucune méthode dédiée, ce qui est cohérent avec leur nature.